End-to-End Data Engineering Pipeline using Azure + Databricks

Name: Vemula Raghunath Goud

Contents

1. Project Overview 2
2. Architecture Diagram 2
3. Technologies Used 3
4. Data Sources 3
5. Data Pipeline Flow 4
6. Data Modeling Approach 13
7. Security Implementation 14
8. Orchestration & Automation 17
9. Cost Optimization 18

1. Project Overview

Project Title
End-to-End Scalable Data Pipeline using Azure Data Factory and Databricks
Objective
Design and implement a **scalable, secure, and production-ready data pipeline** that:

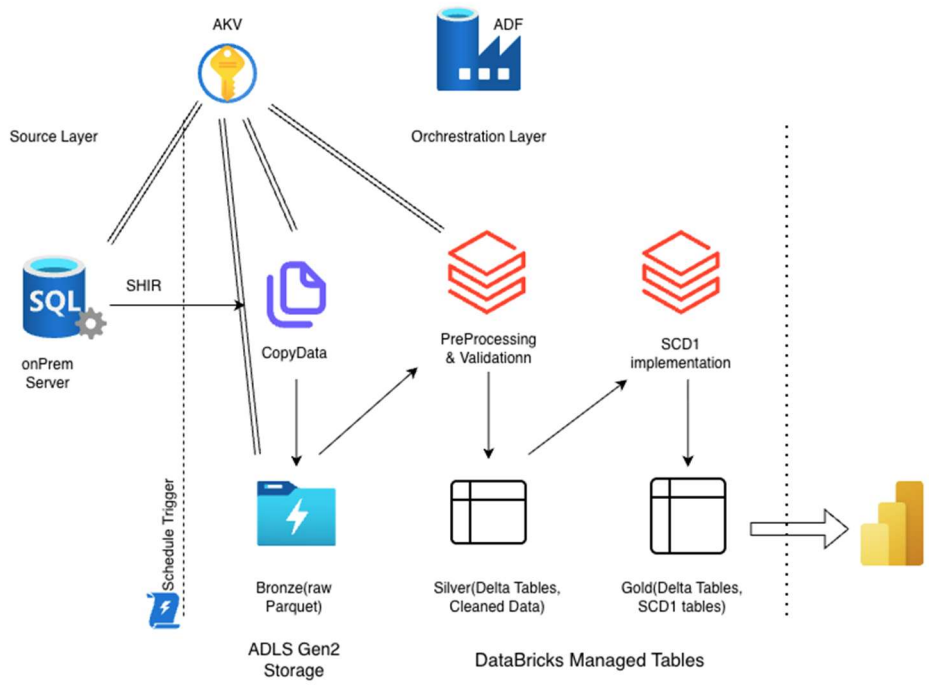
- Ingests structured and semi-structured data
- Processes data using Medallion architecture
- Produces analytics-ready datasets for downstream consumption

Architecture Pattern

Medallion Architecture
Bronze → Silver → Gold

- Metadata-driven ingestion framework
- Event-driven pipeline execution
- Delta Lake-based transformations
- SCD Type 1 implementation using MERGE
- Secure integration using Key Vault + Service Principal

2. Architecture Diagram



Architecture Components

- On-Prem SQL Server (VM): Storage
- Azure Data Lake Storage Gen2 (ADLS)
- Azure Data Factory (ADF): Orchestrations
- Azure Databricks: Transformations
- Azure Key Vault: Security
- Azure SQL DB: Consumption

3. Technologies Used

Layer	Technology
Ingestion	Azure Data Factory
Storage	ADLS Gen2
Compute	Azure Databricks
Processing	PySpark, Delta Lake
Security	Azure Key Vault, Service Principal
Source Systems	On-Prem SQL Server
Orchestration	ADF Pipelines
Target	Delta Tables (Gold Layer)

4. Data Sources

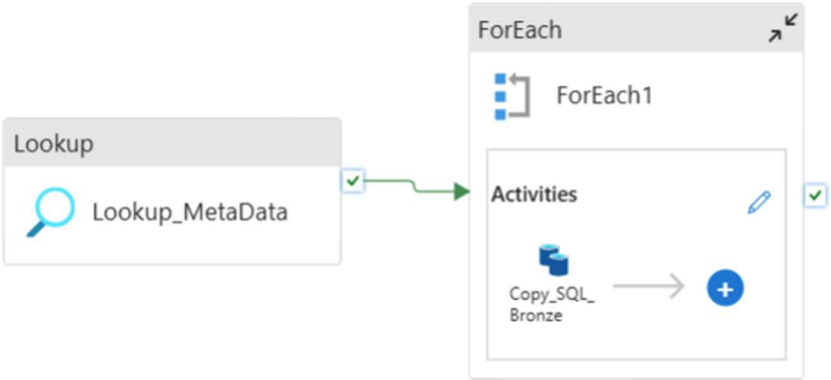
4.1 On-Prem SQL Server (VM)

Tables:

- Customers
- Products
- Orders
- Sales
- Employees

5. Data Pipeline Flow

5.1 Bronze Layer (ADF – Ingestion Layer)



Approach

Metadata-driven ingestion

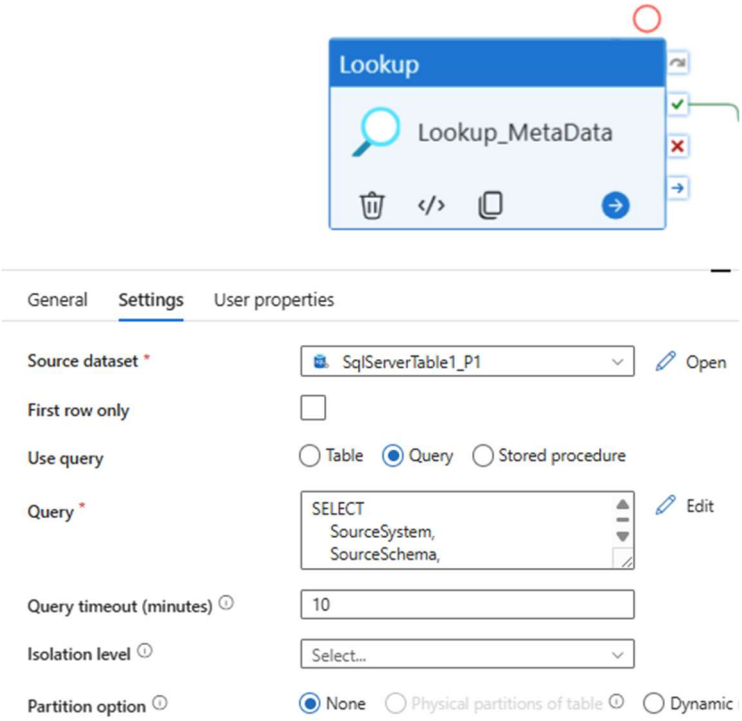
MetaData table from On-Premises serves as the lookup table for driving the ingestion from source to sink, incorporates load type, sequence, source-sink details, watermarking, active status, etc columns.

Dynamic table loading using:

Query example used for the metadata table

```
SELECT
    SourceSystem,
    SourceSchema,
    SourceTable,
    BronzeContainer,
    BronzeFolder,
    BronzeFileName,
    SilverContainer,
    SilverFolder,
    GoldTable,
    LoadType,
    PrimaryKeyColumn,
    WatermarkColumn,
    IsActive,
    LoadSequence
FROM dbo.ADF_Table_Metadata
WHERE IsActive = 1
ORDER BY LoadSequence;
```

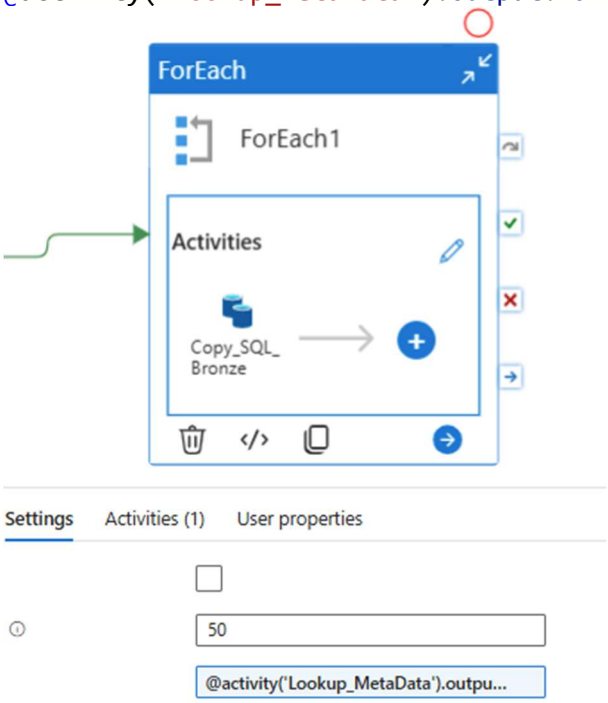
Lookup Activity



ForEach Activity

Iterates through every record in metadata table (one row record info per table).

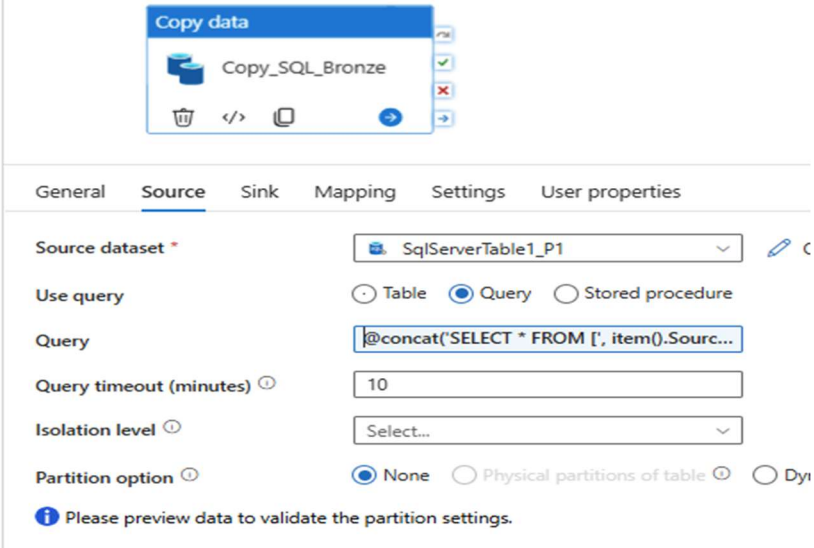
```
@activity('Lookup_MetaData').output.value
```



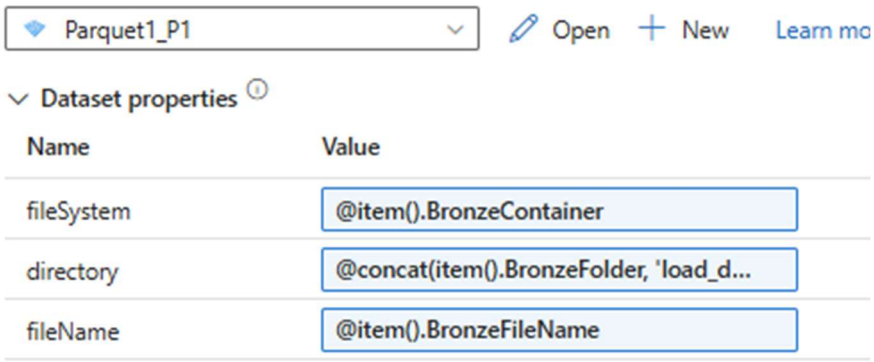
Copy Activity

Using dynamic expression to define source table's schema and table name, SELECT * FROM expression picks up data form the table to be copied (ingested).

```
@concat('SELECT * FROM [', item().SourceSchema, '].[', item().SourceTable,
']')
```



Sink location is driven by MetaData table's fields as well. And stored as parquets.



Folder Structure

bronze/sqlserver/<table_name>/load_date=YYYY-MM-DD/

bronze/api/<endpoint>/load_date=YYYY-MM-DD/

Key Design Decisions

- No transformation at this stage
- Preserve raw data for audit & replay
- Partitioning based on ingestion date

5.2 Silver Layer (Databricks – Processing Layer)

Transformations Applied

- Null validation on primary keys
- Duplicate removal using window functions

```
Yesterday (<1s)                                                    4
# Null / zero PK check
df_clean = df.filter(col("CustomerID").isNotNull() & (col("CustomerID") != 0))
df_clean: pyspark.sql.connect.dataframe.DataFrame = [CustomerID: integer, CustomerName: string ... 2 more fields]

Yesterday (1s)                                                     5
# Adding Timestamp column
df_clean = df_clean.withColumn("processed_ts", current_timestamp())
df_clean: pyspark.sql.connect.dataframe.DataFrame = [CustomerID: integer, CustomerName: string ... 3 more fields]

Yesterday (1s)                                                     6
# Duplicate removal
w = Window.partitionBy("CustomerID").orderBy(col("processed_ts").desc())
df_clean = df_clean.withColumn("rn", row_number().over(w)) \
                .filter(col("rn") == 1) \
                .drop("rn")
df_clean: pyspark.sql.connect.dataframe.DataFrame = [CustomerID: integer, CustomerName: string ... 3 more fields]
```

- Data type casting and format validation

```
Yesterday (<1s)                                                    5
# Date validation
df_clean = df_clean.withColumn("OrderDate", to_date(col("OrderDate")))

df_clean = df_clean.filter(col("OrderDate").isNotNull())
df_clean: pyspark.sql.connect.dataframe.DataFrame = [OrderID: integer, CustomerID: integ
```

- Standardized column naming

```
Yesterday (<1s)

# Remove unnecessary columns / rename columns
df_clean = df_clean.select(
    col("CustomerID").alias("src_CustomerID"),
    col("CustomerName").alias("src_CustomerName"),
    col("City").alias("src_City"),
    col("Country").alias("src_Country"),
    col("processed_ts")
)

df_clean: pyspark.sql.connect.dataframe.DataFrame = [src_Custom
```

Audit columns such as processed_ts added using created_timestamp().

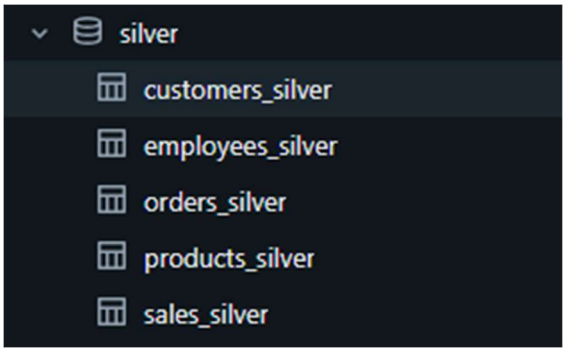
```
Yesterday (<1s)                                                    6

# Processing timestamp
df_clean = df_clean.withColumn("processed_ts", current_timestamp())

df_clean: pyspark.sql.connect.dataframe.DataFrame = [OrderID: integer, CustomerID: in
```

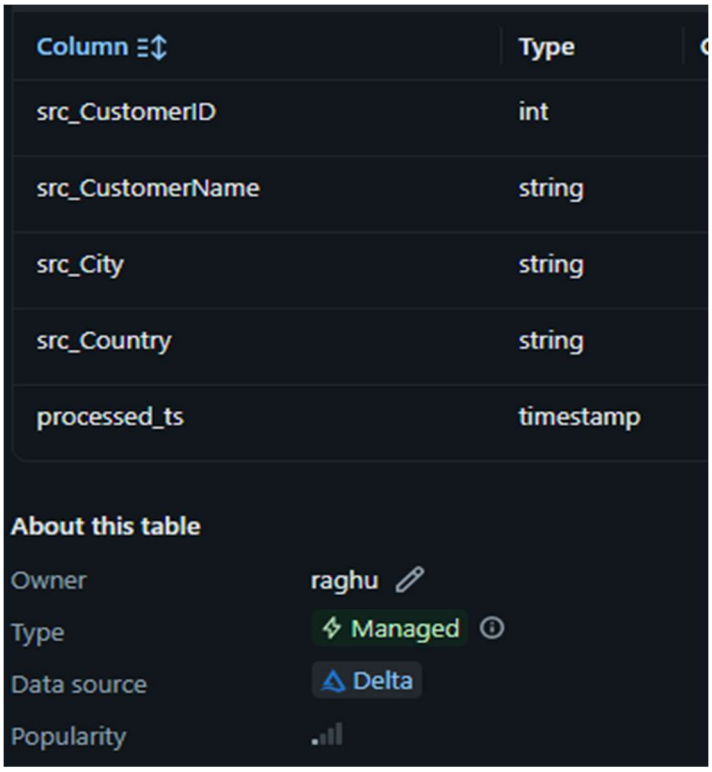
Managed delta tables created using Cluster By optimization prior to writing data.

```
Yesterday (4s)                                                     9

catalog_name = 'adbrag'
schema_name = 'silver'

spark.sql(f"""
CREATE TABLE IF NOT EXISTS {catalog_name}.{schema_name}.orders_silver
(
    src_OrderID INT,
    src_CustomerID INT,
    src_OrderDate DATE,
    src_OrderAmount DECIMAL(10,2),
    processed_ts TIMESTAMP
)
USING DELTA
CLUSTER BY (src_OrderID)
""")
```

Output

- Cleaned Delta Tables (Managed Tables)





Column	Type
src_CustomerID	int
src_CustomerName	string
src_City	string
src_Country	string
processed_ts	timestamp

About this table

Owner	raghu
Type	Managed
Data source	Delta
Popularity	

5.3 Gold Layer (Databricks – Business Layer)

Business Transformations

- Aggregations
- Derived metrics

```python
3

from pyspark.sql.functions import col, when, current_timestamp

silver_df = spark.table(f"{catalog_name}.{src_schema_name}.products_silver")

gold_df = (
    silver_df
    .withColumn(
        "price_band",
        when(col("src_Price") < 100, "Low")
        .when((col("src_Price") >= 100) & (col("src_Price") < 300), "Medium")
        .otherwise("High")
    )
    .withColumn(
        "product_status",
        when(col("src_Price") > 0, "Active").otherwise("Inactive")
    )
    .withColumn("gold_loaded_ts", current_timestamp())
)
```

- Banding: Such as revenue bands, price bands

```python
4

gold_df = (
    silver_df
    .withColumn(
        "order_amount_band",
        when(col("src_OrderAmount") < 100, "Small")
        .when((col("src_OrderAmount") >= 100) & (col("src_OrderAmount") < 500), "Medium")
        .otherwise("Large")
    )
    .withColumn(
        "order_day_type",
        when(dayofweek(col("src_OrderDate")).isin(1, 7), "Weekend").otherwise("Weekday")
    )
    .withColumn("order_year", year(col("src_OrderDate")))
    .withColumn("order_month", month(col("src_OrderDate")))
    .withColumn("gold_loaded_ts", current_timestamp())
)
```

Categorization of order values as buckets for quantity and revenue.

```
gold_df = (
    silver_df
    .withColumn(
        "quantity_bucket",
        when(col("src_Quantity") <= 2, "Low")
        .when((col("src_Quantity") > 2) & (col("src_Quantity") <= 5), "Medium")
        .otherwise("High")
    )
    .withColumn(
        "revenue_bucket",
        when(col("src_TotalAmount") < 200, "Low Revenue")
        .when((col("src_TotalAmount") >= 200) & (col("src_TotalAmount") < 1000), "Medium Revenue")
        .otherwise("High Revenue")
    )
    .withColumn("gold_loaded_ts", current_timestamp())
)
```

```
silver_df = spark.table(f"{catalog_name}.{src_schema_name}.products_silver")

gold_df = (
    silver_df
    .withColumn(
        "price_band",
        when(col("src_Price") < 100, "Low")
        .when((col("src_Price") >= 100) & (col("src_Price") < 300), "Medium")
        .otherwise("High")
    )
    .withColumn(
        "product_status",
        when(col("src_Price") > 0, "Active").otherwise("Inactive")
    )
    .withColumn("gold_loaded_ts", current_timestamp())
)
```

SCD Implementation Customers → SCD Type 1

Technique

Hash-based change detection

Logic

- Compute MD5 hash for key columns
- Compare with existing hash
- Perform:
    - UPDATE → if changed
    - INSERT → if new

```python
Yesterday (1s)

src_df = (
    silver_df
    .select(
        col("src_CustomerID"),
        col("src_CustomerName"),
        col("src_City"),
        col("src_Country"),
        col("processed_ts")
    )
    .withColumn(
        "src_hash",
        crc32( #md5
            concat_ws(
                "|",
                coalesce(col("src_CustomerName"), lit("")),
                coalesce(col("src_City"), lit("")),
                coalesce(col("src_Country"), lit(""))
            )
        )
    )
)
```

- Hash-based change detection
- Delta MERGE used

```python
MERGE INTO {catalog_name}.{target_schema_name}.customers_gold_scd1 AS tgt
USING vw_customers_scd1_source AS src
ON tgt.src_CustomerID = src.src_CustomerID

WHEN MATCHED AND tgt.src_hash <> src.src_hash THEN
  UPDATE SET
    tgt.src_CustomerName = src.src_CustomerName,
    tgt.src_City = src.src_City,
    tgt.src_Country = src.src_Country,
    tgt.src_hash = src.src_hash,
    tgt.processed_ts = src.processed_ts,
    tgt.gold_updated_ts = current_timestamp()

WHEN NOT MATCHED THEN
  INSERT
  (
    src_CustomerID,
```

```
  src_CustomerName,
  src_City,
  src_Country,
  src_hash,
  processed_ts,
  gold_updated_ts
)
VALUES
(
  src.src_CustomerID,
  src.src_CustomerName,
  src.src_City,
  src.src_Country,
  src.src_hash,
  src.processed_ts,
  current_timestamp()
)
```

6. Data Modeling Approach

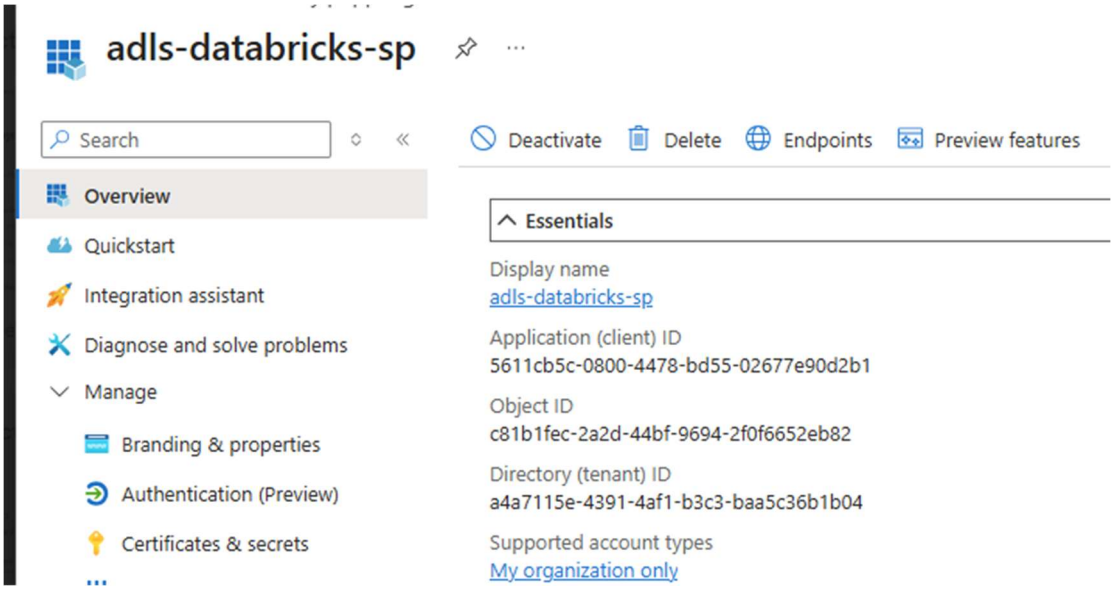
Layer	Description
Bronze	Raw ingestion (schema-on-read)
Silver	Cleaned, validated (schema enforced)
Gold	Business-ready, aggregated

7. Security Implementation

Authentication

- Azure Entra ID Service Principal created for Databricks and storage account.

Application registered in Microsoft Entra ID.

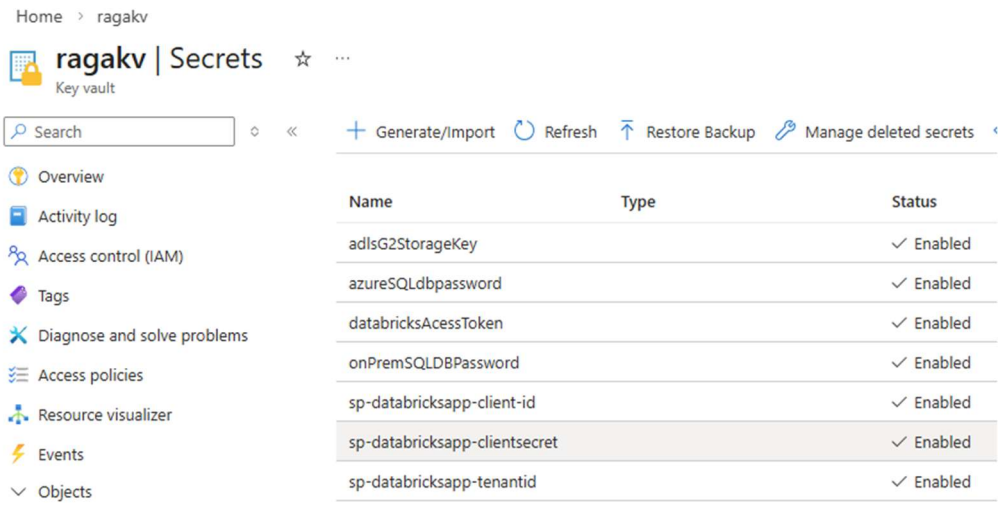


App secret created.

Description	Expires	Value	Secret ID
databricks-secret	13/09/2026	a6w**********	60701e26-c40d-4efe-975e-966f4277e986

Secrets Management:

Secrets created in key vault for required IDs and Client App secret.

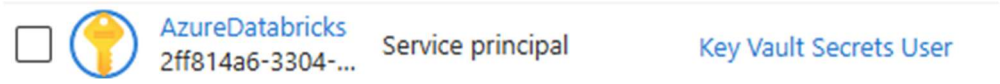


Stored in Azure Key Vault:

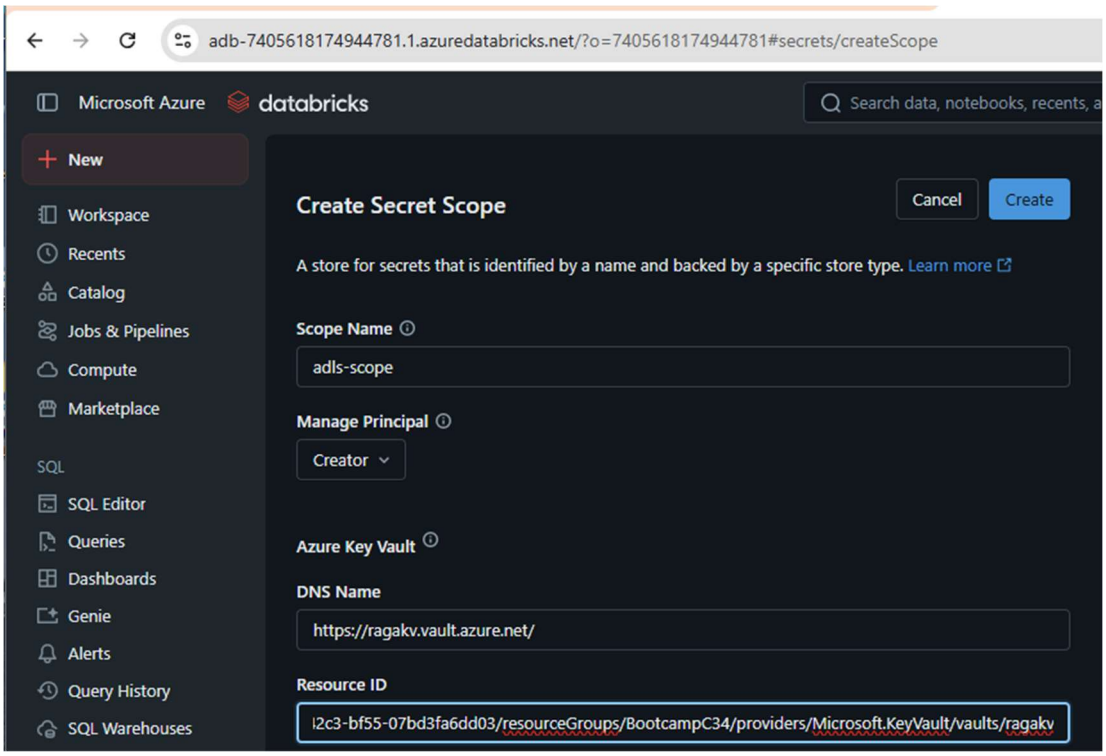
- client-id
- client-secret
- tenant-id

Databricks Integration

AzureDatabricks application role assigned as Key Vault Secrets User on Key Vault Account.



- Key Vault-backed secret scope



- OAuth authentication Connection Notebook used for connecting ADLS Storage account with DataBricks cluster's session

OAuth Connection Notebook

Yesterday (1s) 2

```
dbutils.secrets.get("adls-scope", "sp-databricksapp-client-id")
dbutils.secrets.get("adls-scope", "sp-databricksapp-clientsecret")
dbutils.secrets.get("adls-scope", "sp-databricksapp-tenantid")
```

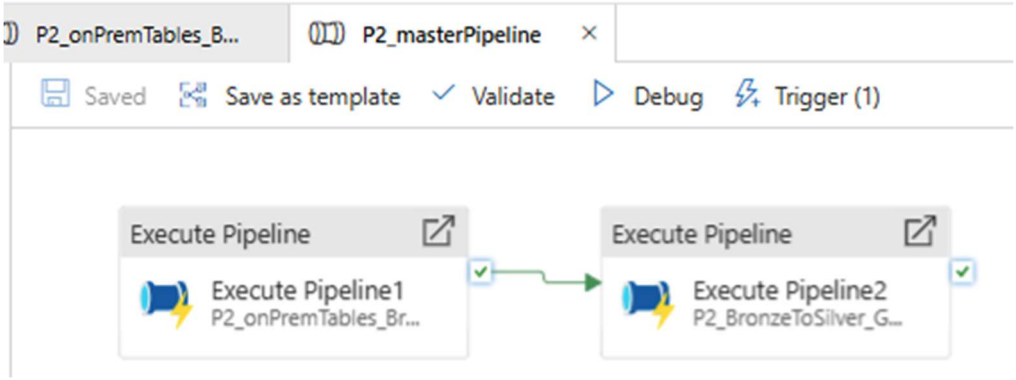
```
'[REDACTED]'
```

Yesterday (3s) 3

```
storage_account = "adlsg2rag"

client_id = dbutils.secrets.get("adls-scope", "sp-databricksapp-client-id")
client_secret = dbutils.secrets.get("adls-scope", "sp-databricksapp-clientsecret")
tenant_id = dbutils.secrets.get("adls-scope", "sp-databricksapp-tenantid")

spark.conf.set(
    f"fs.azure.account.auth.type.{storage_account}.dfs.core.windows.net",
    "OAuth"
)
spark.conf.set(
    f"fs.azure.account.oauth.provider.type.{storage_account}.dfs.core.windows.net",
    "org.apache.hadoop.fs.azurebfs.oauth2.ClientCredsTokenProvider"
)
spark.conf.set(
    f"fs.azure.account.oauth2.client.id.{storage_account}.dfs.core.windows.net",
    client_id
)
spark.conf.set(
    f"fs.azure.account.oauth2.client.secret.{storage_account}.dfs.core.windows.net",
    client_secret
)
spark.conf.set(
    f"fs.azure.account.oauth2.client.endpoint.{storage_account}.dfs.core.windows.net",
    f"https://login.microsoftonline.com/{tenant_id}/oauth2/token"
)
```

8. Orchestration & Automation

ADF Pipelines



- Ingestion pipeline
- Transformation pipeline

Trigger Types

- Scheduled triggers: Ingests OnPrem tables to Raw ADLS Gen2 container

Design Pattern

- Metadata-driven
- Parameterized pipelines

Databricks Execution

- Notebook-based modular design
- Reusable configuration notebook

Every notebook runs connection notebook to establish connection between DataBricks and storage account

```
Yesterday (2s)

%run ./connectionNotebook

[SecretMetadata(key='adlsG2StorageKey'),
 SecretMetadata(key='azureSQLdbpassword'),
 SecretMetadata(key='onPremSQLDBPassword'),
 SecretMetadata(key='sp-databricksapp-client-id'),
 SecretMetadata(key='sp-databricksapp-clientsecret'),
 SecretMetadata(key='sp-databricksapp-tenantid')]

'[REDACTED]'
```

Run notebooks include several notebooks to be run per stage.

```
Yesterday (9s)

%run ./01_customers_bronze_to_silver

[SecretMetadata(key='adlsG2StorageKey'),
 SecretMetadata(key='azureSQLdbpassword'),
 SecretMetadata(key='onPremSQLDBPassword'),
 SecretMetadata(key='sp-databricksapp-client-id'),
 SecretMetadata(key='sp-databricksapp-clientsecret'),
 SecretMetadata(key='sp-databricksapp-tenantid')]

'[REDACTED]'

[FileInfo(path='abfss://bronze@adlsg2rag.dfs.core.windows
 FileInfo(path='abfss://bronze@adlsg2rag.dfs.core.windows
6000),
 FileInfo(path='abfss://bronze@adlsg2rag.dfs.core.windows

> df: pyspark.sql.connect.dataframe.DataFrame = [CustomerID:

root
 |-- CustomerID: integer (nullable = true)
 |-- CustomerName: string (nullable = true)
 |-- City: string (nullable = true)
 |-- Country: string (nullable = true)

> df_clean: pyspark.sql.connect.dataframe.DataFrame = [Custom
> df_clean: pyspark.sql.connect.dataframe.DataFrame = [Custom
> df_clean: pyspark.sql.connect.dataframe.DataFrame = [Custom
> df_clean: pyspark.sql.connect.dataframe.DataFrame = [src_Cus

DataFrame[]

Yesterday (8s)

%run ./02_products_bronze_to_silver
```

9. Cost Optimization

- Auto-terminate Databricks clusters
- Use job clusters instead of all-purpose for running notebooks from ADF pipeline.

- Optimize storage using Delta compression
- Partition pruning for queries
- Avoid unnecessary data movement

Please find project related scripts and json configs under dev and adf_publish.

https://github.com/raghu091994/azureProjects

Includes:

- Pipeline JSON files
- SQL scripts
- Architecture diagram
- Project documentation